

Massive Graph Management & Analytics

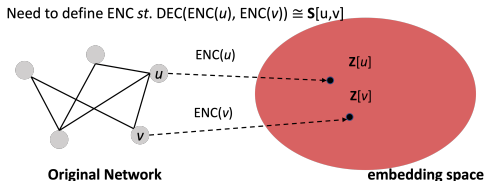
GRAPH NEURAL NETWORK

Nacéra Seghouani

Computer Science Department, CentraleSupélec
Laboratoire Interdisciplinaire des Sciences du Numérique, LISN
nacera.seghouani@centralesupelec.fr

2024-2025

Motivation & Challenges

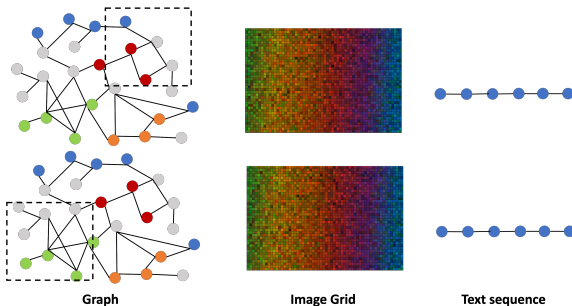


- 👉 The node embedding approaches discussed before use a shallow embedding approach to generate representations of nodes
- 👉 Encoder is just an embedding lookup. Inherently *transductive*
 - ✓ Every node has its own embedding.
 - ✓ Do not incorporate node features.
 - ✓ No parameters shared between nodes.
 - ✓ Cannot generate embeddings for nodes that are not seen during training

Motivation & Challenges

- ☞ Focus on more complex encoder models. Graph Neural Networks (GNNs)   is a general framework for defining deep neural networks on graph data.
- ☞ The key idea is to learn representations of nodes that actually depend on the structure of the graph, as well as any feature information we might have.
- ☞ Tasks:
 - ✓ Node classification
 - ✓ Link prediction
 - ✓ Community detection (Identify densely linked clusters of nodes)
 - ✓ Network similarity (How similar are two (sub)networks)
- ☞ The primary challenge is related to how to define complex encoders for graph-structured data.
- ☞ Usual deep learning such as Convolutional Neural Networks (CNNs) well-defined for grid-structured inputs (ex. images) and Recurrent Neural Networks (RNNs) well-defined for sequences (ex. text) are not relevant for graph data.
- ☞ Networks are far more complex
 - ✓ Arbitrary size and complex topological structure (i.e., no spatial locality like grids)
 - ✓ No fixed node ordering or reference point
 - ✓ Often dynamic and have multimodal features

Motivation & Challenges



- ☞ There no fixed notion of window sliding or locality on the graph
- ☞ Need to define a new deep learning architecture: $ENC(v)$ is multiple layers of non linear transformations based on graph structure

Deep Neural Network Basics

- Loss function: $\text{Min}_{\theta} \mathcal{L}(y, f_{\theta}(x))$
where f is a linear layer or an MLP or neural network (eg. GNN), x a minibatch sample of input data x (supervised)
Pick θ that minimises the loss
- Forward propagation: Compute $\mathcal{L}$ given x (epoch=forward pass)
- Back-propagation: Compute/estimate gradient $\nabla_{\theta} \mathcal{L}$ using a chain rule. Update θ
- Stochastic gradient descent (SGD) to minimise $\mathcal{L}$ for weights θ over many iterations.

Permutation Invariance and Equivariance

- ☞ One naive approach to define a deep neural network over graphs would be to simply use the adjacency matrix as input to a deep neural network.
 - ✓ For example, to generate an embedding of an entire graph we could simply flatten the adjacency matrix and feed the result to a multi-layer perceptron (MLP):
 $\mathbf{z}_{\mathcal{G}} = \text{MLP}(\mathbf{A}_1 \oplus \mathbf{A}_2 \oplus \dots \oplus \mathbf{A}_{|V|})$ where $\mathbf{A}_i \in R^{|V|}$ denotes a row of the adjacency matrix. $\oplus$ is vector concatenation (don't confuse with direct sum)
- ☞ The issue is that depends on the order of nodes in $\mathbf{A}$.

Permutation Invariance and Equivariance

- ☞ In mathematical terms, any function f that takes $(\mathbf{A}, \mathbf{X})$ as input (resp. adjacency matrix and node features vector) should ideally satisfy one of the two following properties:

- ✓ Permutation Invariance: permute the input, the output stays the same

$$f(\mathbf{PAP}^T, \mathbf{PX}) = f(\mathbf{A}, \mathbf{X})$$

- ✓ Permutation Equivariance: permute the input, output also permutes accordingly

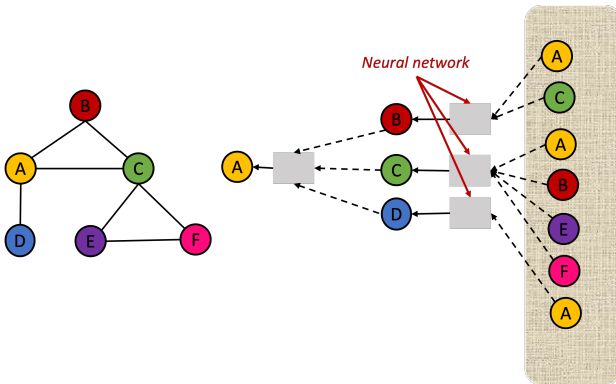
$$f(\mathbf{PAP}^T, \mathbf{PX}) = \mathbf{P}f(\mathbf{A}, \mathbf{X})$$

where $\mathbf{P}$ is a permutation matrix.

- ☞ The shallow encoders discussed are examples of permutation equivariant functions.
- ☞ Graph neural networks consist of multiple permutation equivariant / invariant functions.

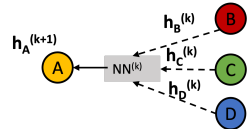
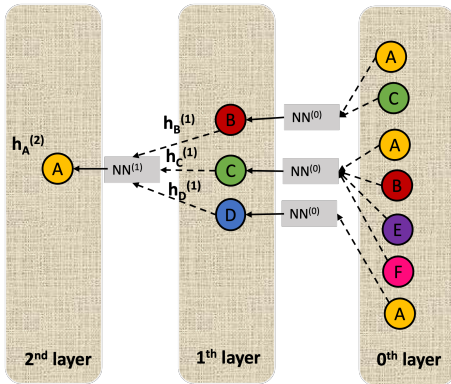
Problem Definition

- Generate node embeddings based on local neighbourhoods
- Aggregate information from neighbours using neural network



Problem Definition

- 👉 Model of arbitrary depth
- 👉 Layer- k embeddings gets information from nodes that are k hops away



Neural Message Passing

Design graph neural networks that are permutation invariant / equivariant by passing and aggregating information from neighbours

- ☞ Recursively update the node features by aggregating information from their neighbouring nodes.
 - ✓ The process performs series of message passing steps, where each node sends a message to its neighbors, which are then aggregated to produce an updated feature representation for each node.
 - ✓ At each iteration, every node aggregates information from its local neighborhood, and as these iterations progress each node embedding contains more and more information from further reaches of the graph.
 - ✓ After iteration After k iterations every node embedding contains information about its k -hop neighborhood.
- ☞ But what kind of “information” do these node embeddings actually encode? Generally, this information comes in two forms:
 - ✓ On the one hand there is structural information about the graph. For example, after k iterations of GNN (k) message passing, the embedding $\mathbf{h}_u$ of node u might encode information about the degrees of all the nodes in u 's k -hop neighborhood.
 - ✓ The other key kind of information captured by GNN node embedding is feature- based. Each node also encode information about all the features in their k -hop neighborhood, and possibly edge representations.

Neural Message Passing

- A single node aggregates messages from its local neighborhood. $\mathbf{h}_u^k$ is the node embedding of u at k -th layer

$$\mathbf{h}_u^{(k+1)} = \mathbf{g}^{(k)} \left(\mathbf{h}_u^{(k)}, \mathbf{f}^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\}) \right) = \mathbf{g}^{(k)} \left(\mathbf{h}_u^{(k)}, m_u^{(k)} \right)$$

where $\mathbf{f}^{(k)}$ and $\mathbf{g}^{(k)}$ are resp. aggregation and transformation (update) function at k -layer, differentiable functions (i.e., neural networks) and $m_u^{(k)}$ is the message that is aggregated from u 's $\mathcal{N}(u)$.

- ✓ The initial embeddings at $k = 0$ are set to the input features for all the nodes, i.e., $\mathbf{h}_u^0$. After running K iterations of the GNN message passing, we can use the output of the final layer to define the embeddings for each node, i.e., $\mathbf{z}_u = \mathbf{h}_u^K, \forall u \in V$
- ✓ For ex., rich node features to use are gene expression features in biological networks or text features in social networks. However, where no node features there are several options using node statistics using metrics discussed before.

The basic GNN message passing

$$\mathbf{h}_u^{(k)} = \sigma \left(W_{self}^{(k)} \mathbf{h}_u^{(k-1)} + W_{neigh}^{(k)} \sum_{v \in \mathcal{N}_u} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)} \right)$$

where $W_{self}, W_{neigh} \in \mathbb{R}^{d \times d}$ are trainable parameter matrices and σ denotes an element wise non-linearity (e.g., a tanh or ReLU). The bias term $\mathbf{b}^{(k)} \in \mathbb{R}^d$ (omitted for notation simplicity)

Basic GNN:

$$m_u^{(k)} = \sum_{v \in \mathcal{N}_u} \mathbf{h}_v^{(k)}$$

$$g(\mathbf{h}_u, m_u) = \sigma \left(W_{self} \mathbf{h}_u + W_{neigh} m_u \right)$$

$$\mathbf{H}^{(k)} = \sigma \left(\mathbf{H}^{(k-1)} W_{self}^{(k)} + \mathbf{A} \mathbf{H}^{(k-1)} W_{neigh}^{(k)} \right) \text{ where } \mathbf{H}^{(k)} \in \mathbb{R}^{|V| \times d}$$

The basic GNN message passing

Message Passing with Self-loops: add to the neighbours the node itself for aggregation. No need to define an explicit update, as it is implicitly included in the aggregation.

$$m_u = \sum_{v \in \mathcal{N}_u} \mathbf{h}_v^{(k)}$$

Adding self-loops is equivalent to sharing parameters between the W_{self} and W_{neigh} matrices:

$$\mathbf{H}^{(k)} = \sigma \left(\mathbf{H}^{(k-1)} W^{(k)} + \mathbf{A} \mathbf{H}^{(k-1)} W^{(k)} \right) \text{ where } \mathbf{H}^{(k)} \in \mathbb{R}^{|V| \times d}$$

Simplifying the message passing alleviates overfitting, but also limits the GNN expressivity, as the information coming from the node's neighbours cannot be differentiated from the one coming from the node itself.

Simplified GCN

- Need to normalize

$$m_u^{(k)} = \frac{1}{|\mathcal{N}_u|} \sum_{v \in \mathcal{N}_u} \mathbf{h}_v^{(k)}$$

- Symmetric normalization

$$m_u^{(k)} = \sum_{v \in \mathcal{N}_u} \frac{\mathbf{h}_v^{(k)}}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_v|}}$$

$$\mathbf{h}^{(k)} = \sigma \left(W^{(k)} \sum_{v \in \mathcal{N}_u} \frac{\mathbf{h}_v^{(k)}}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_v|}} \right)$$

- More sophisticated aggregation: Set pooling, ...
- Neighbourhood attention strategy to improve the aggregation by assigning an attention weight or importance/influence to each neighbour.

R-GCN Multi-relational Graphs

- Commonly known approach is Relational Graph Convolutional Network (R-GCN, Schlichtkrul et al, 2017):
- augment the aggregation to accommodate multiple relation types by specifying a separate transformation matrix per relation type:

$$m_u^{(k)} = \sum_{r \in R} \sum_{v \in \mathcal{N}_u(r)} \frac{W_r \mathbf{h}_u}{f_{\text{norm}}(\mathcal{N}_u, \mathcal{N}_v)}$$

f_{norm} is normalisation function, depends on the neighbourhood of u as well as v being aggregated over. Several normalisation strategies analogous to those discussed earlier.

- Drawback: The number of parameters to learn (one matrix to learn per relation) increases
⇒ overfitting on rare relations and slow learning.
⇒ Basis-decomposition and block-diagonal-decomposition used for regularising the weights of R-GCN-layers as a linear combination of basis transformations

$$W_r = \sum_{i=1}^b \alpha_{i,r} \mathbf{B}_i$$

$(\mathbf{B}_1, \dots, \mathbf{B}_b)$ is a tensor formed by the basis matrices, $\alpha_r = (\alpha_{1,r}, \dots, \alpha_{b,r})$ is a vector containing the basis combination weights for relation r

a separate aggregation parameter per relation, is applicable for discrete edge features. Otherwise we can leverage these features in attention or by concatenating this information with the neighbour embeddings

Loss Function for Node Classification

- ☞ The standard way for node classification task is to train GNNs in a fully-supervised manner, where we define the loss using a softmax classification function and negative log-likelihood loss:

$$\mathcal{L} = \sum_{u \in V_{train}} -\log(\text{softmax}(\mathbf{z}_u, \mathbf{y}_u))$$

where V_{train} is training dataset, and $\mathbf{y}_u \in \mathbb{N}^c$ is one-hot vector indicating the class (from 1 to c classes) node $u \in V_{train}$

$$\text{softmax}(\mathbf{z}_u, \mathbf{y}_u) = \sum_{i=1}^c \mathbf{y}_u[i] \frac{e^{\mathbf{z}_u^T \mathbf{w}_i}}{\sum_{j=1}^c e^{\mathbf{z}_u^T \mathbf{w}_j}} \text{ where } \mathbf{w} \in \mathbb{R}^{d \times c} \text{ are trainable parameters}$$

- ☞ Binary cross entropy is calculated on top of sigmoid outputs. Categorical cross-entropy (for multi-class) is calculated over softmax activation outputs.
- ☞ V_{train} nodes are included in the GNN message passing operations, and they are also used to compute the loss.

Loss Function for Node Classification

- ☞ V_{train} training nodes are included in GNN message passing operations, and they are also used to compute the loss.
- ☞ V_{trans} transductive nodes, unlabelled nodes $\notin V_{train}$, not used in the final layer embeddings to compute the loss function. But these nodes and their edges are involved in GNN message passing, GNN will generate their hidden representation.
- ☞ V_{induc} inductive nodes not used in either the loss computation (unlabelled) or GNN message passing (their edges are not aggregated while GNN is trained).

Loss Function for Link Prediction

- Many applications: Recommender systems (social networks, online sale, ...), Knowledge Graph (KG) completion, ... in various domains.
- Pairwise node embedding loss functions such as those discussed in shallow embedding approaches (neighbourhood reconstruction losses) with the output of GNNs.
- Contrastive learning: original and corrupted pairs of connected nodes (head or tail of triples).
Example of loss

$$\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} \text{Max}(0, -\mathbf{z}_u^T \mathbf{z}_v + \mathbf{z}_u^T \mathbf{z}_{\tilde{v}} + \delta)$$

where $\mathcal{D}$ is the set of training pairs/triples, $\tilde{v}$ is the corrupted tail and δ is how much larger should positive pair similarity be compared to negative one.

Some Main Issues

- 👉 Modelling level: The input graph & task
 - How to define the graph, which features from text, images, statistics such as commons neighbours, katz, clustering coefficient, PageRank, ...
 - How to define similarity
- 👉 Importance pooling
 - Define Importance pooling: Define importance-based neighbourhoods, for ex., by simulating random walks and selecting the neighbours with the highest visit counts.
 - Pool over the chosen nodes that are not necessarily neighbours.
- 👉 The challenge is that we cannot simply run message passing on a subset of the nodes in a graph without losing information.
 - No guarantee that selecting a random subset of nodes will lead to a connected graph, and selecting a random subset of nodes for each mini-batch can reduce model performances.
 - First select a set of target nodes for a batch and then recursively sample the neighbours of these nodes in order to ensure the connectivity.

Some Main Issues

👉 Training

- Hyperparameters: Dimension, Loss parameters, ... → Look at learning curves, monitor gradient update ratios, ...
- Generate samples, synthetic data, visualise learned representations embeddings using nearest neighbours for instance
- Error analysis, examples where the model is failing

👉 Efficiency Concerns

- Avoid the possibility of sampling too many nodes for a batch: subsample the neighbours of each node, using a fixed size to limit the memory footprint. Many subsampling strategies defined in the literature.
- Perform a localised graph convolution around a specific node, does not need the entire graph during the training. Efficient MapReduce inference.
- Many repeated computation. Need to avoid repeated computation. Avoided through careful engineering to ensure the embedding for each node u in the batch is computed at most once.

- 👉 Pre-train a GNN to reconstruct missing edges before fine-tuning on a node classification loss. Studies show that randomly initialised GNN is equally strong compared to one pre-trained on a neighbourhood reconstruction loss.
GNN message passing already effectively encodes neighbourhood information.

GNN Scalability Issues

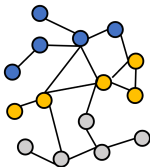
- ✎ **Width:** In k layer GNNs, one node aggregates information from $O(N^k)$ nodes where N is the average number of neighbours per node.
 - Hub nodes who are connected to a huge number of nodes.
 - Should we aggregate all neighbours? Which neighbours should we aggregate messages from?
 - Limit the neighbourhood by sampling a fixed number of neighbours
 - Random sampling: Assign same sampling probabilities to all neighbours.
 - Importance sampling: Assign different sampling probabilities to all neighbours FastGCN, LADIES, AS-GCN, GCN-BS.

GNN Scalability Issues

- ☞ **Depth:** How many hops should we explore? When the depth k increases GNNs face neighbours explosion $O(N^k)$
- Over-smoothing: GNNs become deep, nodes share many neighbours
 - Node embeddings become indistinguishable
 - PairNorm, to keep the same distance across layers and remove not connected pairs
 - The size of the neighbourhood could be compressed into a fixed size
 - Decoupling the two concepts of depths in GNNs:
 - (i) Neighbourhood that each node aggregates information from
 - (ii) Number of layers in GNNs (2 or 3 commonly used)

Setting up GNN Prediction Tasks

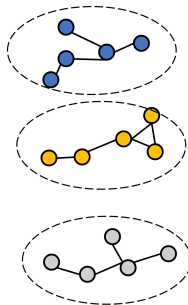
- 👉 How to split the graph into train, validation and test datasets? Not easy nodes are not independent.
 - ✓ Training dataset to optimize GNN parameters (learn representation)
 - ✓ Validation dataset to tune the model hyperparameters
 - ✓ Test dataset to evaluate the model performances for different tasks
- 👉 Pre-defined split or random split using different random seeds the average performances are reported
- 👉 Two ways:
 - ✓ Transductive: One input graph. The embeddings are computed for the whole graph.
 - ✓ Inductive: Multiple input graphs



Training

Validation

Test



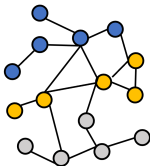
Setting up GNN Prediction Tasks

👉 Transductive Learning:

- ✓ At the training time the embeddings are computed for the whole graph, trained, validated and tested on different subsets of node labels. All the splits can observe the entire graph structure. Only a subset of labels are observed (semi-supervised). Applicable for node or link predictions tasks.

👉 Inductive Learning:

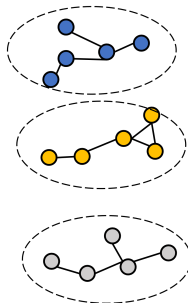
- ✓ Training, validation and test are performed on different graphs on different subsets of node labels. Only a part of the graph structure is observed -> better for graph prediction task.



Training

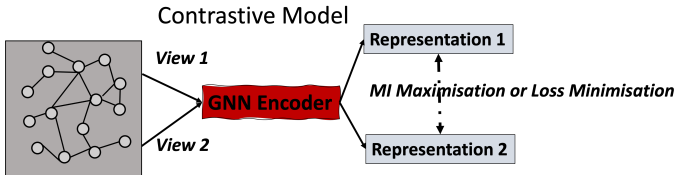
Validation

Test



Graph Contrastive Learning (GCL)

- ☞ The main idea of contrastive learning is to make representations agree with each other under proper transformations:
- ✓ Node/edge dropping: missing edges or nodes do not alter semantics.
 - ✓ Edge perturbation: randomly adding or dropping a certain ratio of edges → semantic robustness against connectivity variations.
 - ✓ Attribute masking. The underlying assumption is that missing partial vertex attributes does not affect the model predictions much → semantic robustness against losing partial attributes, context information
 - ✓ Subgraph sampled using random walk for ex. It assumes that the semantics of the graph can be much preserved in its (partial) local structure.



Graph Embedding to Detect Anomalies in Videos

- High spatio-temporal complexity existing in real-world scenarios.
- Ano-Graph: Learning Normal Scene Contextual Graphs to Detect Video Anomalies (Pourreza et al., 2021) 
 - ✓ Learning and modelling the interaction of normal objects. A Spatio-Temporal Graph (STG) is made by considering each node as an object's feature extracted from a real-time off-the-shelf object detector, and edges are made based on their interactions.

